第28课 二分查找

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：基础二分查找（查找目标值）

题目：给定一个从小到大排好序的数组，和一个要查找的数 x。如果 x 在数组中，输出它的下标（从 0 开始）；如果不在，输出 -1。

输入样例：

```
10 23
2 5 8 12 16 23 38 42 56 72
```

输出样例：

```
5
```

讲解思路：

1. 先读入数组（已经有序，不需要排序）。
2. 设定 `left = 0`，`right = n - 1`。
3. 循环：只要 `left <= right`，就继续找。
4. 算中间位置 `mid = (left + right) / 2`。
5. 比较 `a[mid]` 和 `x`：相等就输出 mid；小了就往右找；大了就往左找。
6. 循环结束还没找到，输出 -1。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n, x;
    cin >> n >> x;

    int a[100005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    // 二分查找
    int left = 0, right = n - 1;
    int ans = -1;  // 默认没找到

    while (left <= right) {
        int mid = (left + right) / 2;

        if (a[mid] == x) {
            ans = mid;   // 找到了!
            break;       // 跳出循环
        } else if (a[mid] < x) {
            left = mid + 1;   // x 在右半边，左手右移
        } else {
            right = mid - 1;  // x 在左半边，右手左移
        }
    }

    cout << ans << endl;
    return 0;
}
```

逐行带读要点：

- `left = 0, right = n - 1`：一开始整个数组都是查找范围。
- `while (left <= right)`：只要左手没有跑到右手右边，就还有数没看过。
- `mid = (left + right) / 2`：取中间位置（整数除法，自动向下取整）。
- `left = mid + 1`：为什么要 `+1`？因为 `mid` 这个位置已经看过了，不用再看了！同理 `right = mid - 1`。

例题 2：二分查找——找不到的情况 & 统计次数

题目：给定一个有序数组和 q 个查询，对于每个查询的数 x，输出它在数组中的下标（从 1 开始），如果不存在输出 `0`。同时统计每次查找比较了多少次。

输入样例：

```
8 3
1 3 5 7 9 11 13 15
5 10 13
```

输出样例：

```
3 3
0 4
7 3
```

讲解思路：

1. 这题有多个查询，用一个循环处理每次查询。
2. 每次查找前把计数器清零，每比较一次就加 1。
3. 注意题目下标从 1 开始，所以找到后输出 `mid + 1`。
4. 通过输出比较次数，让学生直观感受二分查找的效率。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n, q;
    cin >> n >> q;

    int a[100005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    while (q--) {
        int x;
        cin >> x;

        int left = 0, right = n - 1;
        int ans = 0;   // 0 表示没找到（题目下标从1开始）
        int cnt = 0;   // 比较次数计数器

        while (left <= right) {
            int mid = (left + right) / 2;
            cnt++;  // 每次比较，计数器+1

            if (a[mid] == x) {
                ans = mid + 1;  // 下标从1开始
                break;
            } else if (a[mid] < x) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        cout << ans << " " << cnt << endl;
    }

    return 0;
}
```

课堂互动：

> 同学们看，数组有 8 个数，我们最多比较了几次？——**4 次！** 如果用顺序查找呢？最多要 **8 次**。数据越多，差距越大！

例题 3（进阶）：在有序数组中找第一个 ≥ x 的位置

题目：给定一个从小到大排好序的数组（可能有重复元素），找出第一个大于等于 x 的元素的下标（从 1 开始）。如果所有元素都小于 x，输出 `n + 1`。

输入样例：

```
8 5
1 2 4 4 5 5 7 9
```

输出样例：

```
5
```

讲解思路：

> 这道题和前面不太一样。前面是找"等于 x 的数"，这道题是找"第一个大于等于 x 的数"。
>
> 比喻：你去超市买苹果，预算是 5 块钱。货架上的苹果按价格从低到高排好了：1元、2元、4元、4元、5元、5元、7元、9元。你想找到**第一个你买得起的苹果**（价格 ≥ 5 的第一个）。

关键变化：

- 当 `a[mid] >= x` 时，mid 可能是答案，但左边可能还有更靠前的，所以**记下来但继续往左找**（`right = mid - 1`）。
- 当 `a[mid] < x` 时，mid 肯定不是答案，往右找（`left = mid + 1`）。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n, x;
    cin >> n >> x;

    int a[100005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    int left = 0, right = n - 1;
    int ans = n + 1;  // 默认：所有数都比 x 小

    while (left <= right) {
        int mid = (left + right) / 2;

        if (a[mid] >= x) {
            ans = mid + 1;      // 记下这个位置（下标从1开始）
            right = mid - 1;    // 但继续往左找，看有没有更靠前的
        } else {
            left = mid + 1;     // 太小了，往右找
        }
    }

    cout << ans << endl;
    return 0;
}
```

对比强调：

	例题 1（找等于 x）	例题 3（找第一个 ≥ x）
找到相等时	直接 `break`	**不 break**，记下答案继续往左找
初始 ans	-1	n + 1
适用场景	查一个数在不在	查位置 / 有重复元素

> 同学们，例题 3 是二分查找的"升级版"，竞赛里特别常用。今天能听懂就很棒了，不用着急一下子全掌握！

附：例题源代码与运行结果

例题 1：Basic Binary Search

```
// Given a sorted array (ascending) and a target x,
// output the index (0-based) of x, or -1 if not found.
#include <iostream>
using namespace std;

int main() {
    int n, x;
    cin >> n >> x;

    int a[100005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    // binary search
    int left = 0, right = n - 1;
    int ans = -1;  // default: not found

    while (left <= right) {
        int mid = (left + right) / 2;

        if (a[mid] == x) {
            ans = mid;        // found it
            break;
        } else if (a[mid] < x) {
            left = mid + 1;   // x is in the right half
        } else {
            right = mid - 1;  // x is in the left half
        }
    }

    cout << ans << endl;
    return 0;
}
```

Sample Input

```
10 23
2 5 8 12 16 23 38 42 56 72
```

Expected Output

```
5
```

例题 2：Binary Search with Comparison Count

```
// Given a sorted array and q queries, for each query x,
// output its 1-based index (or 0 if not found) and the
// number of comparisons made during the search.
#if 0
#include <iostream>
using namespace std;

int main() {
    int n, q;
    cin >> n >> q;

    int a[100005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    while (q--) {
        int x;
        cin >> x;

        int left = 0, right = n - 1;
        int ans = 0;   // 0 means not found (1-based indexing)
        int cnt = 0;   // comparison counter

        while (left <= right) {
            int mid = (left + right) / 2;
            cnt++;  // count each comparison

            if (a[mid] == x) {
                ans = mid + 1;  // convert to 1-based index
                break;
            } else if (a[mid] < x) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        cout << ans << " " << cnt << endl;
    }

    return 0;
}
#endif
```

Sample Input

```
8 3
1 3 5 7 9 11 13 15
5 10 13
```

Expected Output

```
3 3
0 4
7 3
```

例题 3：Find First Element >= x

```
// Given a sorted array (may have duplicates),
// find the 1-based index of the first element >= x.
// If all elements are less than x, output n+1.
#if 0
#include <iostream>
using namespace std;

int main() {
    int n, x;
    cin >> n >> x;

    int a[100005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    int left = 0, right = n - 1;
    int ans = n + 1;  // default: all elements < x

    while (left <= right) {
        int mid = (left + right) / 2;

        if (a[mid] >= x) {
            ans = mid + 1;      // record this position (1-based)
            right = mid - 1;    // keep searching left for earlier one
        } else {
            left = mid + 1;     // too small, search right
        }
    }

    cout << ans << endl;
    return 0;
}
#endif
```

Sample Input

```
8 5
1 2 4 4 5 5 7 9
```

Expected Output

```
5
```